Full Adder using Two Half Adders

STRUCTURAL

entity: full_adder

sum = a ⊕ b ⊕ cin | cout = (a·b) + (cin·(a⊕b))

Adds three 1-bit numbers using TWO half_adder components plus one OR gate. HA1 adds a,b → sum1 + carry1. HA2 adds sum1,cin → final sum + carry2. The two carries are OR'd for cout. Pure structural (component instantiation) architecture.

▸ LIVE SIMULATOR

sum = a XOR b XOR cin | cout = c1 OR c2

▸ TRUTH TABLE

a	b	cin	sum	cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

▸ VHDL — FULL_ADDER.VHD (STRUCTURAL)

COPY

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity full_adder is
  Port (
    a    : in  STD_LOGIC;
    b    : in  STD_LOGIC;
    cin  : in  STD_LOGIC;
    sum  : out STD_LOGIC;
    cout : out STD_LOGIC
  );
end full_adder;

architecture Structural of full_adder is

component half_adder is
  Port (a : in  STD_LOGIC;
        b : in  STD_LOGIC;
        s : out STD_LOGIC;
        c : out STD_LOGIC);
end component;

signal s1, c1, c2 : STD_LOGIC;

begin
  HA1: half_adder port map(a, b, s1, c1);
  HA2: half_adder port map(s1, cin, sum, c2);
  cout <= c1 or c2;
end Structural;
```

▸ SUB-COMPONENT — HALF_ADDER.VHD (MUST COMPILE FIRST)

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity half_adder is
  Port (
    a : in  STD_LOGIC;
    b : in  STD_LOGIC;
    s : out STD_LOGIC;
    c : out STD_LOGIC
  );
end half_adder;

architecture Behavioral of half_adder is
begin
  s <= a xor b;
  c <= a and b;
end Behavioral;
```

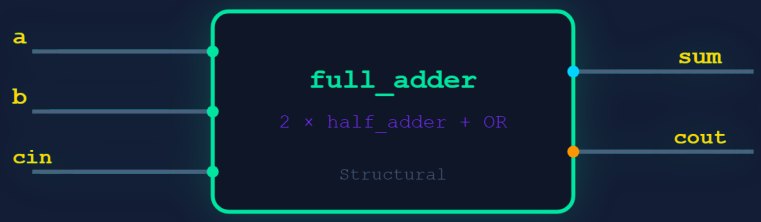
HA1(a,b) → s1 = a⊕b, c1 = a·b

HA2(s1,cin) → sum = s1⊕cin, c2 = s1·cin

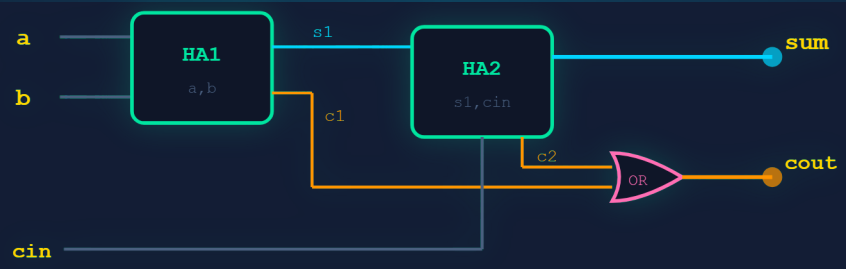
OR → cout = c1 + c2

COMPILE ORDER: 1. half_adder.vhd → 2. full_adder.vhd

▸ BLOCK DIAGRAM — FULL_ADDER



▸ STRUCTURAL CIRCUIT — HA1, HA2, OR



HA1:a,b→s1,c1 | HA2:s1,cin→sum,c2 | cout=c1 OR c2

▸ TESTBENCH — FULL_ADDER_TB.VHD

COPY

```
ENTITY full_adder_tb IS END full_adder_tb;
ARCHITECTURE behavior OF full_adder_tb IS
  COMPONENT full_adder
    PORT(a,b,cin : IN  std_logic;
         sum,cout : OUT std_logic);
  END COMPONENT;
  signal a,b,cin : std_logic := '0';
  signal sum,cout : std_logic;
BEGIN
  uut: full_adder PORT MAP(a,b,cin,sum,cout);
  process begin
    a<='0';b<='0';cin<='0';wait for 100 ns;
    a<='0';b<='0';cin<='1';wait for 100 ns;
    a<='0';b<='1';cin<='0';wait for 100 ns;
    a<='0';b<='1';cin<='1';wait for 100 ns;
    a<='1';b<='0';cin<='0';wait for 100 ns;
    a<='1';b<='0';cin<='1';wait for 100 ns;
    a<='1';b<='1';cin<='0';wait for 100 ns;
    a<='1';b<='1';cin<='1';wait for 100 ns;
    wait;
  end process;
END;
```

▸ EXECUTION STEPS — XILINX ISE

1 **New Project**
→ File → New Project → full_adder_2ha , HDL: VHDL

2 **Create half_adder**
→ New Source → VHDL Module → half_adder (a,b,s,c)

3 **Enter HA code**
→ s<=a xor b; c<=a and b;

4 **Create full_adder**
→ component half_adder + signals s1,c1,c2

5 **Port maps**
→ HA1(a,b,s1,c1); HA2(s1,cin,sum,c2); cout<=c1 or c2

6 **Check Syntax**
→ Synthesize-XST → 0 errors

7 **Testbench**
→ VHDL Test Bench → all 8 input combos

8 **Simulate**
→ ISim → Run 800 ns → verify sum/cout

EXAM NOTE: STRUCTURAL — full adder instantiates half_adder TWICE + OR gate.
half_adder.vhd compiled before full_adder.vhd.

Full Adder using Half Adder

BEHAVIORAL

entity: full_adder_bh

sum = a ⊕ b ⊕ cin | cout = ab + bcin + acin

Same full adder logic derived from the half-adder equations, written in a single behavioral/dataflow file — direct concurrent signal assignments, no component instantiation. sum reuses the half-adder XOR; cout is the majority function.

▸ LIVE SIMULATOR

sum = a XOR b XOR cin | cout = (a AND b) OR (b AND cin) OR (a AND cin)

▸ TRUTH TABLE

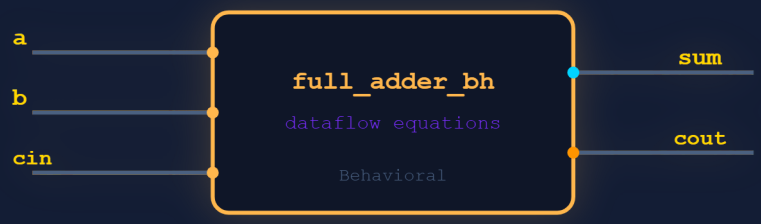
a	b	cin	sum	cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

VHDL — full_adder_bh.vhd (Behavioral)

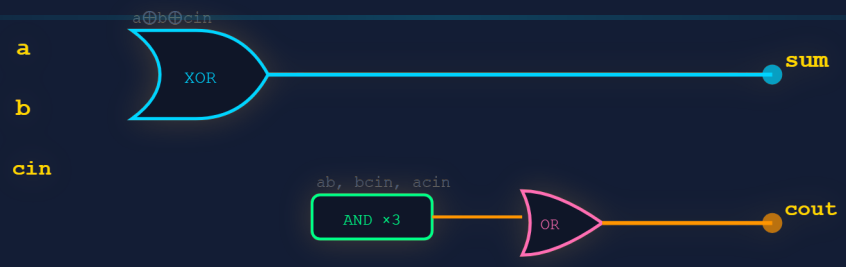
COPY

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity full_adder_bh is
  Port (
    a    : in  STD_LOGIC;
    b    : in  STD_LOGIC;
    cin  : in  STD_LOGIC;
    sum  : out STD_LOGIC;
    cout : out STD_LOGIC
  );
end full_adder_bh;

architecture Behavioral of full_adder_bh is
begin
  sum  <= a xor b xor cin;
  cout <= (a and b) or (b and cin) or (a and cin);
end Behavioral;
```

Block Diagram — full_adder_bh



Gate-Level Circuit



sum = 3-input XOR | cout = OR of 3 AND terms (majority)

▸ TESTBENCH — FULL_ADDER_BH_TB.VHD

COPY

```
ENTITY full_adder_bh_tb IS END full_adder_bh_tb;
ARCHITECTURE behavior OF full_adder_bh_tb IS
  COMPONENT full_adder_bh
    PORT(a,b,cin : IN  std_logic;
         sum,cout : OUT std_logic);
  END COMPONENT;
  signal a,b,cin : std_logic := '0';
  signal sum,cout : std_logic;
BEGIN
  uut: full_adder_bh PORT MAP(a,b,cin,sum,cout);
  process begin
    a<='0';b<='0';cin<='0';wait for 100 ns;
    a<='0';b<='0';cin<='1';wait for 100 ns;
    a<='0';b<='1';cin<='0';wait for 100 ns;
    a<='0';b<='1';cin<='1';wait for 100 ns;
    a<='1';b<='0';cin<='0';wait for 100 ns;
    a<='1';b<='0';cin<='1';wait for 100 ns;
    a<='1';b<='1';cin<='0';wait for 100 ns;
    a<='1';b<='1';cin<='1';wait for 100 ns;
    wait;
  end process;
END;
```

▸ EXECUTION STEPS — XILINX ISE

1 **New Project**
→ File → New Project → full_adder_bh , HDL: VHDL

2 **New Source**
→ VHDL Module → full_adder_bh — ports a,b,cin,sum,cout

3 **Enter Equations**
→ sum<=a xor b xor cin; and cout majority

4 **Check Syntax**
→ Synthesize-XST → 0 errors

5 **Testbench**
→ VHDL Test Bench → all 8 combos, 100 ns each

6 **Simulate**
→ ISim → Run 800 ns → verify

EXAM NOTE: BEHAVIORAL — single file, no components. Concurrent assignments only. Same truth table as the 2-HA structural version.

Full Subtractor using Two Half Subtractors

STRUCTURAL | entity: full_subtractor

diff = a ⊕ b ⊕ bin | bout = a'b + a'bin + b·bin

Subtracts with borrow-in (a − b − bin) using TWO half_subtractor components plus one OR gate. HS1: a,b → d1 + bo1. HS2: d1,bin → diff + bo2. The two borrows are OR'd for bout.

▸ LIVE SIMULATOR

diff = a XOR b XOR bin | bout = bo1 OR bo2

▸ TRUTH TABLE

a	b	bin	diff	bout
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

▸ VHDL — FULL_SUBTRACTOR.VHD (STRUCTURAL)

COPY

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity full_subtractor is
  Port (
    a    : in  STD_LOGIC;
    b    : in  STD_LOGIC;
    bin  : in  STD_LOGIC;
    diff : out STD_LOGIC;
    bout : out STD_LOGIC
  );
end full_subtractor;

architecture Structural of full_subtractor is

component half_subtractor is
  Port (a : in  STD_LOGIC;
        b : in  STD_LOGIC;
        d : out STD_LOGIC;
        bo : out STD_LOGIC);
end component;

signal d1, bo1, bo2 : STD_LOGIC;

begin
  HS1: half_subtractor port map(a, b, d1, bo1);
  HS2: half_subtractor port map(d1, bin, diff, bo2);
  bout <= bo1 or bo2;
end Structural;
```

▸ SUB-COMPONENT — HALF_SUBTRACTOR.VHD (MUST COMPILE FIRST)

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity half_subtractor is
  Port (
    a  : in  STD_LOGIC;
    b  : in  STD_LOGIC;
    d  : out STD_LOGIC;
    bo : out STD_LOGIC
  );
end half_subtractor;

architecture Behavioral of half_subtractor is
begin
  d  <= a xor b;
  bo <= (not a) and b;
end Behavioral;
```

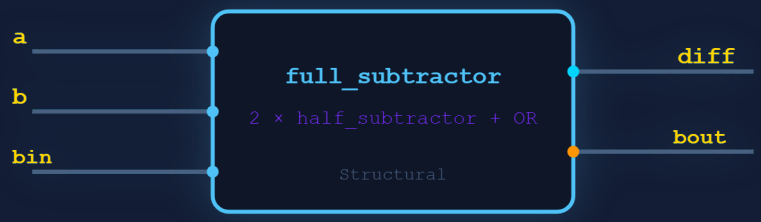
HS1(a,b) → d1 = a⊕b, bo1 = a'·b

HS2(d1,bin) → diff = d1⊕bin, bo2 = d1'·bin

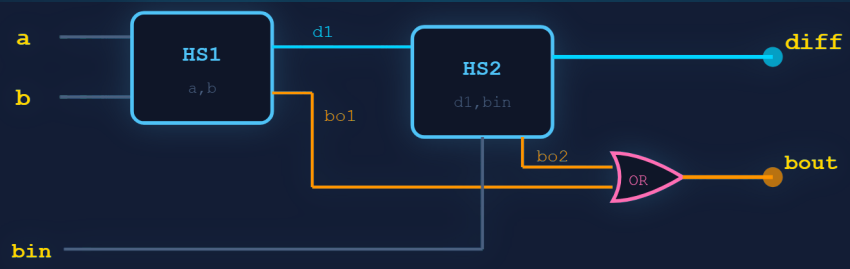
OR → bout = bo1 + bo2

COMPILE ORDER: 1. half_subtractor.vhd → 2. full_subtractor.vhd

▸ BLOCK DIAGRAM — FULL_SUBTRACTOR



▸ STRUCTURAL CIRCUIT — HS1, HS2, OR



HS1:a,b→d1,bo1 | HS2:d1,bin→diff,bo2 | bout=bo1 OR bo2

▸ TESTBENCH — FULL_SUBTRACTOR_TB.VHD

COPY

```
ENTITY full_subtractor_tb IS END full_subtractor_tb;
ARCHITECTURE behavior OF full_subtractor_tb IS
  COMPONENT full_subtractor
    PORT(a,b,bin : IN  std_logic;
         diff,bout : OUT std_logic);
  END COMPONENT;
  signal a,b,bin : std_logic := '0';
  signal diff,bout : std_logic;
BEGIN
  uut: full_subtractor PORT MAP(a,b,bin,diff,bout);
  process begin
    a<='0';b<='0';bin<='0';wait for 100 ns;
    a<='0';b<='0';bin<='1';wait for 100 ns;
    a<='0';b<='1';bin<='0';wait for 100 ns;
    a<='0';b<='1';bin<='1';wait for 100 ns;
    a<='1';b<='0';bin<='0';wait for 100 ns;
    a<='1';b<='0';bin<='1';wait for 100 ns;
    a<='1';b<='1';bin<='0';wait for 100 ns;
    a<='1';b<='1';bin<='1';wait for 100 ns;
    wait;
  end process;
END;
```

▸ EXECUTION STEPS — XILINX ISE

1

New Project

→ File → New Project → full_subtractor_2hs , VHDL

2

Create half_subtractor

→ VHDL Module → half_subtractor (a,b,d,bo)

3

Enter HS code

→ d<=a xor b; bo<=(not a) and b;

4

Create full_subtractor

→ component half_subtractor + signals d1,bo1,bo2

5

Port maps

→ HS1(a,b,d1,bo1); HS2(d1,bin,diff,bo2); bout<=bo1 or bo2

6

Check Syntax

→ Synthesize-XST → 0 errors

7

Testbench + Simulate

→ All 8 combos → ISim Run 800 ns

EXAM NOTE: STRUCTURAL — full subtractor instantiates half_subtractor TWICE + OR gate. HS borrow uses (NOT a) so a' feeds the AND. Compile HS before FS.

Full Subtractor using Half Subtractor

BEHAVIORAL | entity: full_sub_bh

diff = a ⊕ b ⊕ bin | bout = a'b + a'bin + b·bin

Same full subtractor logic from the half-subtractor equations, written as a single behavioral/dataflow file — concurrent assignments, no components. diff reuses the half-subtractor XOR; bout is the sum-of-products borrow.

▸ LIVE SIMULATOR

diff = a XOR b XOR bin | bout = (a' AND b) OR (a' AND bin) OR (b AND bin)

▸ TRUTH TABLE

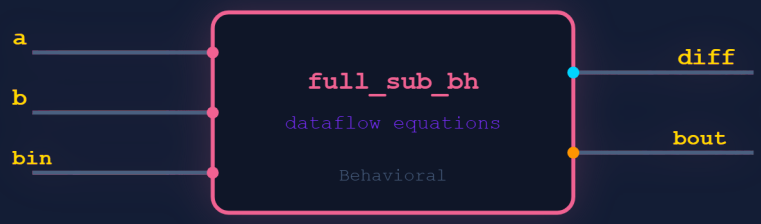
a	b	bin	diff	bout
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

VHDL — full_sub_bh.vhd (Behavioral)

COPY

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity full_sub_bh is
  Port (
    a    : in  STD_LOGIC;
    b    : in  STD_LOGIC;
    bin  : in  STD_LOGIC;
    diff : out STD_LOGIC;
    bout : out STD_LOGIC
  );
end full_sub_bh;

architecture Behavioral of full_sub_bh is
begin
  diff <= a xor b xor bin;
  bout <= ((not a) and b) or ((not a) and bin) or (b and bin);
end Behavioral;
```

Block Diagram — full_sub_bh



Gate-Level Circuit

diff = 3-input XOR | bout = OR of 3 AND terms (a' inverts a)

▸ TESTBENCH — FULL_SUB_BH_TB.VHD

COPY

```
ENTITY full_sub_bh_tb IS END full_sub_bh_tb;
ARCHITECTURE behavior OF full_sub_bh_tb IS
  COMPONENT full_sub_bh
    PORT(a,b,bin : IN  std_logic;
         diff,bout : OUT std_logic);
  END COMPONENT;
  signal a,b,bin : std_logic := '0';
  signal diff,bout : std_logic;
BEGIN
  uut: full_sub_bh PORT MAP(a,b,bin,diff,bout);
  process begin
    a<='0';b<='0';bin<='0';wait for 100 ns;
    a<='0';b<='0';bin<='1';wait for 100 ns;
    a<='0';b<='1';bin<='0';wait for 100 ns;
    a<='0';b<='1';bin<='1';wait for 100 ns;
    a<='1';b<='0';bin<='0';wait for 100 ns;
    a<='1';b<='0';bin<='1';wait for 100 ns;
    a<='1';b<='1';bin<='0';wait for 100 ns;
    a<='1';b<='1';bin<='1';wait for 100 ns;
    wait;
  end process;
END;
```

▸ EXECUTION STEPS — XILINX ISE

1

New Project

→ File → New Project → full_sub_bh , VHDL

2

New Source

→ VHDL Module → full_sub_bh — ports a,b,bin,diff,bout

3

Enter Equations

→ diff<=a xor b xor bin; and bout SOP

4

Check Syntax

→ Synthesize-XST → 0 errors

5

Testbench

→ VHDL Test Bench → all 8 combos

6

Simulate

→ ISim → Run 800 ns → verify diff/bout

EXAM NOTE: BEHAVIORAL — single file, no components. bout = SOP with a' (NOT a). Same truth table as the 2-HS structural version.